

Software Básico



**INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
NORTE DE MINAS GERAIS**

fppt.com

Números reais

- Indo além de inteiros **com** e **sem sinal**, as linguagens de programação admitem **números com frações**, que são chamados reais na matemática.
- Exemplos de números reais:
 - $3,14159265_{10}$ (π)
 - $2,71828_{10}$ (e)
 - $0,000000001_{10}$ (segundos em um nanossegundo)
 - $3.155.760.000,0_{10}$ (segundos em um século)



Números reais

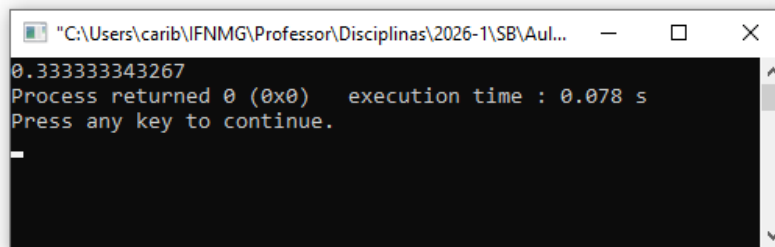
- A primeira questão que aparece ao tratar números **não inteiros** no computador digital **é a perda de precisão ou os erros numéricos**. Por exemplo, mesmo em notação decimal o valor $1/3$ não será representado perfeitamente, ele será registrado com algo semelhante a 0.3333 (se o computador utilizar 4 dígitos).

```
#include <stdio.h>

int main()
{
    float num = 1.0/3;

    printf("%.12f", num);

    return 0;
}
```



```
"C:\Users\carib\IFNMG\Professor\Disciplinas\2026-1\SB\Aul...
0.333333343267
Process returned 0 (0x0)   execution time : 0.078 s
Press any key to continue.
```



Números reais

Converter $18,125_{10}$ para binário.

1. Converter a parte inteira para binário com divisões sucessivas. $18_{10} = 10010_2$
2. Converter a parte fracionária usando múltiplas multiplicações.
 - $0,125_{10} = 001_2$
 - $18,125_{10} = 10010,001_2$



Números reais

Converter $4,1_{10}$ para binário.

1. Converter a parte inteira para binário com divisões sucessivas. $4_{10} = 100_2$
2. Converter a parte fracionária usando múltiplas multiplicações.
 - $0,1_{10} = 00011001100\dots_2$
 - $4,1_{10} = 100,00011001100\dots_2$



Números reais e o *Hardware*

- É impossível representar qualquer número real na máquina.
 - Temos uma infinidade de números reais, e o *hardware* é finito.
- Uma forma de armazenar essas aproximações é através de pontos flutuantes.
- Conceito implementado em grande parte dos processadores comerciais.



Números reais e o *Hardware*

$$\pm S \times B^{\pm E}$$

- Esse número pode ser armazenado em uma palavra binária com três campos:
 - Sinal: mais ou menos.
 - Significando S.
 - Expoente E.



Números reais e o *Hardware*

- ,A notação é chamada **notação científica**, que tem um único dígito à esquerda do ponto decimal.
- Um número na **notação científica** que não tem 0s à esquerda do ponto decimal é chamado de número **normalizado**, que é o modo normal como o escrevemos.
- **Exemplos:**
 - $7.0_{10} \times 10^{-9}$ é normalizado.
 - $0.3_{10} \times 10^{-8}$ não é normalizado.
 - $170.0_{10} \times 10^{-10}$ não é normalizado.



Números reais e o *Hardware*

- Podemos fazer o mesmo com números binários.
- Para manter um **número binário** na forma normalizada, precisamos de uma **base** que possamos **aumentar ou diminuir** exatamente pelo número de *bits* que o número precisa ser deslocado para ter um dígito diferente de zero à esquerda do ponto decimal.

Somente uma base de 2 atende à nossa necessidade.



Números reais e o *Hardware*

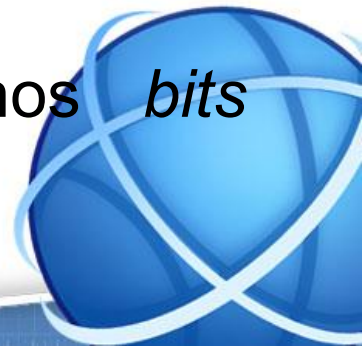
- Para simplificar as operações sobre números de ponto flutuante, normalmente é exigido que eles sejam ***normalizados***.
- Um número **normalizado** é aquele em que o dígito mais significativo do significando é diferente de zero.

$$\pm 1.bbb \dots b \times 2^{\pm E}$$



Números reais e o *Hardware*

- A **base** B é implícita e não precisa ser armazenada;
- Considera-se que o **ponto fracionário** está a direita do *bit* mais à esquerda (ou mais significativo) do significando. Ou seja, existe um *bit* a esquerda do ponto fracionário.
- O *bit* mais à esquerda armazena o **sinal** do número (0 = positivo, 1 = negativo).
- O valor do **expoente** é armazenado nos *bits* seguintes.



Números reais e o *Hardware*

- Exemplo:
 - $1.001_2 \times 2^1$.
- Vamos chamar o ponto decimal de “ponto binário” para a base 2.
- A **aritmética computacional**, que admite tais números, é chamada **ponto flutuante** porque representa os números em que o ponto binário não é fixo, como acontece para os inteiros.



Números reais e o *Hardware*

- **Exemplo:**



- O *bit* mais à esquerda armazena o ***sinal*** do número (0 = positivo, 1 = negativo).
- O valor do ***expoente*** é armazenado nos 8 *bits* seguintes.



Números reais e o *Hardware*

- **Expoente binário:** A representação usada é conhecida como *representação polarizada*.
- Um valor fixo, chamado de polarização, é subtraído do campo para obter o verdadeiro valor do expoente.
- A polarização é igual a $(2^{k-1} - 1)$, onde k é o número de *bits* no **expoente binário**.



Números reais e o *Hardware*

- A parte final da palavra (23 bits, neste caso) é o ***significando***.
- O termo ***mantissa***, as vezes usado no lugar de ***significando***.



Números reais e o *Hardware*

$$\pm 1.bbb \dots b \times 2^{\pm E}$$

- Como o *bit* mais significativo é sempre 1, é desnecessário armazenar esse bit; ao invés disso, **ele é implícito.**
- O campo de 23 *bits* é usado para armazenar um significando de 24 *bits*.



Números reais e o *Hardware*

- Observe que, quando os *bits* de uma representação polarizada são tratados como inteiros sem sinal, as magnitudes relativas dos números não mudam.
- Por exemplo, nas representações polarizada é sem sinal:
 - o maior número é 1111 e
 - o menor número é 0000.



Números reais e o *Hardware*

- Isso não é verdade com a representação por sinal - **magnitude** ou **complemento de dois**.
- Uma vantagem da representação polarizada é que os números de ponto flutuante não negativos podem ser tratados como inteiros para fins de comparação.



Números reais e o *Hardware*

Representação decimal	Representação sinal-magnitude	Representação em complemento de dois	Representação polarizada
+8	–	–	1111
+7	0111	0111	1110
+6	0110	0110	1101
+5	0101	0101	1100
+4	0100	0100	1011
+3	0011	0011	1010
+2	0010	0010	1001
+1	0001	0001	1000
+0	0000	0000	0111
–0	1000	–	–
–1	1001	1111	0110
–2	1010	1110	0101
–3	1011	1101	0100
–4	1100	1100	0011
–5	1101	1011	0010
–6	1110	1010	0001
–7	1111	1001	0000
–8	–	1000	–

Números reais e o *Hardware*

- **Observe as seguintes características:**
 - O sinal é armazenado no primeiro bit da palavra.
 - O primeiro *bit* do verdadeiro significando é sempre 1 e não precisa ser armazenado no campo de significando.
 - O valor 127 é acrescentado ao verdadeiro expoente para ser armazenado no campo de expoente.
 - A base é 2.



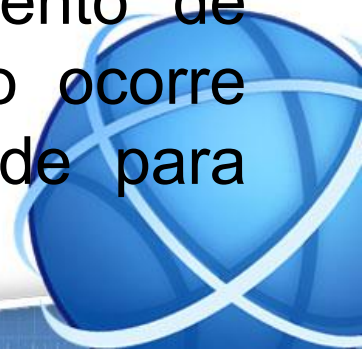
Representação em ponto flutuante

- Um projetista de uma representação em ponto flutuante precisa encontrar um compromisso entre:
 - o tamanho da fração e
 - o tamanho do expoente,
- Pois um tamanho de palavra fixo significa que você precisa tirar um *bit* de um para acrescentar um *bit* ao outro.



Representação em ponto flutuante

- Interrupções por **overflow** podem ocorrer na aritmética de **ponto flutuante** e na aritmética de **inteiros**.
- Observe que **overflow** aqui significa que o expoente é muito grande para ser representado no campo de expoente.
- Para distinguir do **overflow** positivo, o evento de **underflow** é para negativos. Essa situação ocorre quando o expoente negativo é muito grande para caber no campo de expoente.



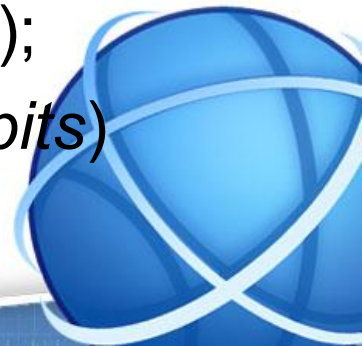
Representação em ponto flutuante

- Uma maneira de reduzir as chances de ***underflow*** ou ***overflow*** é oferecer outro formato que tenha um expoente maior.
 - O **ponto flutuante** com **precisão simples** é o nome do formato anterior.
 - As operações sobre ***doubles*** são indicadas como aritmética de **ponto flutuante** com **precisão dupla**;
 - **S** ainda é o sinal do número, **expoente** é o valor do campo de expoente em 11 bits, e **fração** é o número de 52 bits na fração.



Representação em ponto flutuante

- **aumentar o tamanho da fração** melhora a precisão da fração, enquanto
- **aumentar o tamanho do expoente** aumenta o intervalo de números que podem ser representados.
- Dois formatos para números de ponto flutuante:
 - precisão simples: $e=8$, $m=23$ (total: 32 *bits*) ;
 - precisão dupla: $e=11$, $m = 52$ (total: 64 *bits*);
 - precisão estendida: $e=15$, $m = 63$ (total 80 *bits*)



Padrão de ponto flutuante IEEE 754

- Esses formatos vão além da arquitetura MIPS RISC. Eles fazem parte do padrão de ponto flutuante **IEEE 754**, encontrado em praticamente todo computador inventado desde 1980.
- Esse padrão melhorou bastante tanto na facilidade de portar programas de ponto flutuante quanto a qualidade da aritmética computacional.



Padrão de ponto flutuante IEEE 754

- Para colocar ainda mais *bits* no significando, o IEEE 754 deixa implícito o *bit* 1 inicial dos números binários normalizados.
- Para ser exato, usamos o termo:
 - **significando** a fim de representar o número de 24 ou 53 bits que é 1 mais a fração, e
 - **fração** quando queremos dizer o número de 23 ou 52 bits.

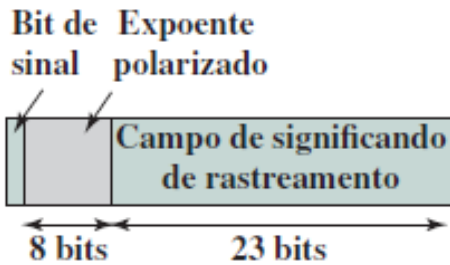


Padrão de ponto flutuante IEEE 754

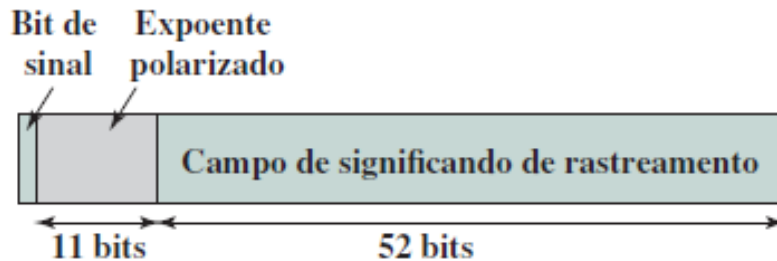
- Esses três formatos binários básicos tem tamanhos em bits de 32, 64 e 128 bits:
 - 24 bits de largura na precisão simples (1 + 23 bits) e
 - 53 bits de extensão na precisão dupla (1 + 52 bits).
 - 113 bits de extensão na precisão dupla estendida (1 + 112).



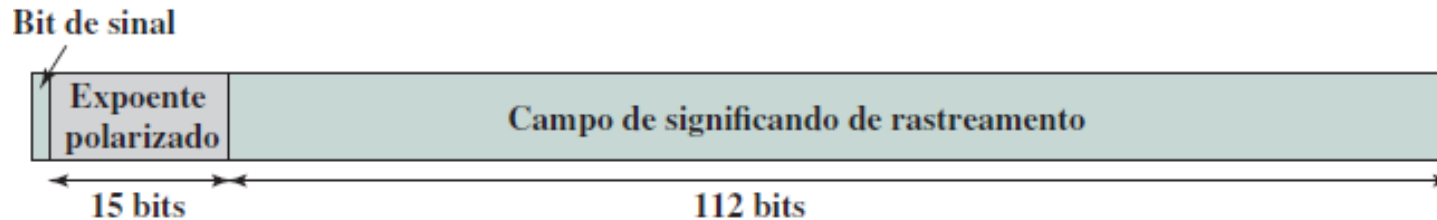
Padrão de ponto flutuante IEEE 754



(a) Formato binário32



(b) Formato binário64



(c) Formato binário128



Padrão de ponto flutuante IEEE 754

Parâmetro	Formato		
	Binário32	Binário64	Binário128
Largura de armazenamento (bits)	32	64	128
Largura de expoente (bits)	8	11	15
Polarização de expoente	127	1023	16383
Expoente máximo	127	1023	16383
Expoente mínimo	-126	-1022	-16382
Intervalo numérico normal aproximado (base 10)	$10^{-38}, 10^{+38}$	$10^{-308}, 10^{+308}$	$10^{-4932}, 10^{+4932}$
Largura do significando de rastreamento (bits)*	23	52	112
Número de expoentes	254	2046	32766
Número de frações	2^{23}	2^{52}	2^{112}
Número de valores	$1,98 \times 2^{31}$	$1,99 \times 2^{63}$	$1,99 \times 2^{128}$
Menor número normal positivo	2^{-126}	2^{-1022}	2^{-16382}
Maior número normal positivo	$2^{128} - 2^{104}$	$2^{1024} - 2^{971}$	$2^{16384} - 2^{16271}$

Padrão de ponto flutuante IEEE 754

- Como 0 não possui um 1 inicial, ele recebe o valor de expoente reservado 0, de modo que o *hardware* não lhe acrescente um 1 inicial.
- Assim, $00\dots00_{\text{bin}}$ representa 0; a representação do restante dos números usa a forma de antes, com o 1 oculto sendo acrescentado:
 - $(-1)^S \times (1 + \text{Fração}) \times 2^E$



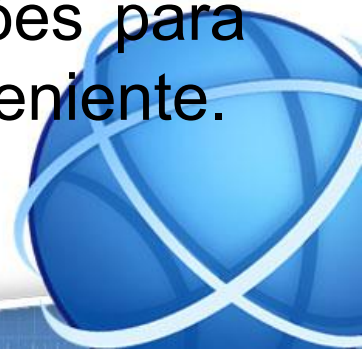
Padrão de ponto flutuante IEEE 754

- Outros recursos do IEEE 754 são símbolos especiais para representar eventos incomuns.
- Por exemplo, em vez de interromper em uma divisão por 0, o software pode definir o resultado para um padrão de *bits* que represente $+\infty$ ou $-\infty$; o maior expoente é reservado a esses símbolos especiais.



Padrão de ponto flutuante IEEE 754

- O IEEE 754 possui um símbolo para o resultado de operações inválidas, como:
 - 0/0, ou
 - a subtração entre infinito e infinito.
- Esse símbolo é **NaN**, de **Not-a-Number** (não é um número).
- A finalidade dos **NaNs** é permitir que os programadores adiem alguns testes e decisões para outro momento no programa, quando for conveniente.



Padrão de ponto flutuante IEEE 754

```
#include <stdio.h>
#include <math.h>
/*
INFINITY
-INFINITY
*/
int main()
{
    float x;
    x = 1.0/0;
    printf("x = %f\n",x);
    x = -1.0/0;
    printf("x = %f\n",x);
    x = INFINITY - INFINITY;
    printf("x = %f\n",x);
    return 0;
}
```



Padrão de ponto flutuante IEEE 754

- Os projetistas do IEEE 754 também queriam uma representação de ponto flutuante que pudesse ser facilmente processada por comparações de inteiros, especialmente para ordenação.
 - *Bit* de sinal é o mais significativa: maior que 0 ou menor que 0;
 - expoente antes do significando simplifica a ordenação dos números de ponto flutuante usando instruções de comparação de inteiros;

Representação decimal	Representação polarizada
+8	1111
+7	1110
+6	1101
+5	1100
+4	1011
+3	1010
+2	1001
+1	1000
+0	0111
-0	-
-1	0110
-2	0101
-3	0100
-4	0011
-5	0010
-6	0001
-7	0000
-8	-



Padrão de ponto flutuante IEEE 754

- Nem todos os padrões de bits nos formatos do IEEE são interpretados pelo modo normal; em vez disso, alguns padrões de bits são usados para representar valores especiais.
- A tabela a seguir indica os valores atribuídos a diversos padrões de bit.
- Os valores de expoente extremos de todos zeros (0) e todos uns (1) (255 em formato simples, 2.047 em formato duplo) definem valores especiais. As classes de números a seguir são representadas:



Padrão de ponto flutuante IEEE 754

- Uma operação de ponto flutuante pode produzir uma destas condições:
 - **Overflow de expoente:** um expoente positivo excede o valor máximo possível para expoente. Em alguns sistemas, isso pode ser designado como $+\infty$ ou $-\infty$.
 - **Underflow de expoente:** um expoente negativo é menor que o valor mínimo possível para expoente (por exemplo, -200 é menor que -127). Isso significa que o número é muito pequeno para ser representado, e pode ser informado como 0.



Padrão de ponto flutuante IEEE 754

- Uma operação de ponto flutuante pode produzir uma destas condições:
 - ***Underflow* de significando:** no processo de alinhamento dos significandos, os dígitos podem sair pela extremidade direita do significando.
 - ***Overflow* de significando:** a adição de dois significandos com o mesmo sinal pode resultar em um *carry* pelo *bit* mais significativo.



Padrão de ponto flutuante IEEE 754

(a) formato binário32

	Sinal	Expoente polarizado	Fração	Valor
zero positivo	0	0	0	0
zero negativo	1	0	0	-0
mais infinito	0	tudo 1	0	∞
menos infinito	1	tudo 1	0	$-\infty$
NaN silencioso	0 ou 1	tudo 1	$\neq 0$; primeiro bit = 1	qNaN
NaN sinalizador	0 ou 1	tudo 1	$\neq 0$; primeiro bit = 0	sNaN
normal positivo diferente de zero	0	$0 < e < 225$	f	$2_e^{-127}(1,f)$
normal negativo diferente de zero	1	$0 < e < 225$	f	$-2_e^{-127}(1,f)$



Padrão de ponto flutuante IEEE 754

(b) formato binário64

	Sinal	Expoente polarizado	Fração	Valor
zero positivo	0	0	0	0
zero negativo	1	0	0	-0
mais infinito	0	tudo 1	0	∞
menos infinito	1	tudo 1	0	$-\infty$
NaN silencioso	0 ou 1	tudo 1	$\neq 0$; primeiro bit = 1	qNaN
NaN sinalizador	0 ou 1	tudo 1	$\neq 0$; primeiro bit = 0	sNaN
normal positivo diferente de zero	0	$0 < e < 2047$	f	$2_e^{-1023}(1,f)$
normal negativo diferente de zero	1	$0 < e < 2047$	f	$-2_e^{-1023}(1,f)$



Padrão de ponto flutuante IEEE 754

(c) formato binário128

	Sinal	Expoente polarizado	Fração	Valor
zero positivo	0	0	0	0
zero negativo	1	0	0	-0
mais infinito	0	tudo 1	0	∞
menos infinito	1	tudo 1	0	$-\infty$
NaN silencioso	0 ou 1	tudo 1	$\neq 0$; primeiro bit = 1	qNaN
NaN sinalizador	0 ou 1	tudo 1	$\neq 0$; primeiro bit = 0	sNaN
normal positivo diferente de zero	0	tudo 1	f	$2^{e-16383}(1,f)$
normal negativo diferente de zero	1	tudo 1	f	$-2^{e-16383}(1,f)$



Referência

